

UAV Dual-Phone System Architecture Guide

System Framework Documentation

May 29, 2026

1 Off-Grid Hotspot Data Transmission

Transmitting data between two devices connected directly via a local Wi-Fi hotspot does not require an active internet connection or cellular mobile data. The hotspot-broadcasting device functions entirely as a local network router. All packet transmission occurs locally within this decentralized local area network (LAN).

2 Workload Topology & Hardware Breakdown

- **Onboard Phone (System-on-Chip / SOC):** Handles high-throughput local processing tasks including real-time computer vision processing, sensor fusion, object tracking, and direct hardware communication with the UAV flight controller. The physical connection to the autopilot board is established over a USB On-The-Go (OTG) cable running binary MAVLink data streams.
- **Ground Phone (Ground Control Station / GCS):** Handles visual representation and high-level strategy. Core tasks include rendering the User Interface (UI), processing manual pilot touch-screen stick inputs, and executing heavy path planning algorithms (such as A* or RRT*).

[Ground Phone: UI/Path Planning]

(Local Wi-Fi Hotspot Link / UDP)

[Onboard Phone: SOC/Vision] (USB OTG / MAVLink) [UAV Flight Controller]

3 ROS 2 System Simulation Framework

ROS 2 utilizes a fully decentralized peer-to-peer Data Distribution Service (DDS), completely removing the legacy centralized `roscore` master node required in ROS 1. This architecture mirrors local Wi-Fi hotspot behavior perfectly, providing natural network healing properties if connections briefly drop.

- **Flight Simulation (SITL):** Use PX4 or ArduPilot Software In The Loop (SITL) bound to a Gazebo 3D simulation environment. The ROS 2 `MAVRos` package bridges simulated telemetry parameters seamlessly to your control scripts.
- **Data Link Simulation (NetEm):** Isolate Ground and Onboard nodes using separate Linux Network Namespaces. Use the NetEm kernel utility to inject predictable wireless latency and packet drops directly into your virtual interface to see how your code handles connection degradation.

Listing 1: NetEm Network Emulation Command Example

```
# Example command to inject 20ms of latency and a 2% packet loss rate
into a simulated node
$ ip netns exec onboard_ns tc qdisc add dev veth_peer root netem delay
  20ms 5ms loss 2%
```

4 Physical Deployment & DDS Constraints

- **Multicast Blockage:** Standard Android and iOS mobile operating systems heavily restrict or completely block multicast network packets while operating in hotspot mode to conserve system battery drain. Because default DDS discovery depends on multicast to find nearby nodes, your applications will fail to discover each other over a physical mobile hotspot.
- **Architectural Solution:** Force your ROS 2 environment configuration to use explicit **Unicast (Peers List)** topology. You must bypass automated discovery and explicitly list your ground station's static target IP address (which defaults to the hotspot gateway address, commonly 192.168.43.1 on Android).

5 C++ Low-Latency UDP Socket Implementation

For direct local communication bypassing ROS 2, standard C++ POSIX sockets provide the lowest overhead possible. Below is a lightweight framework implementation for sending and receiving telemetry structs across the mobile hotspot network interface.

Listing 2: Cross-Phone UDP Telemetry Bridge (C++)

```
#include <iostream>
#include <cstring>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

// Packed telemetry structure to ensure uniform spacing over the wire
struct __attribute__((__packed__)) TelemetryData {
    uint32_t packet_index;
    float roll;
    float pitch;
    float yaw;
    double gps_lat;
    double gps_lon;
};

class TelemetrySocket {
private:
    int sockfd;
    struct sockaddr_in target_addr;
    struct sockaddr_in local_addr;

public:
    TelemetrySocket() : sockfd(-1) {}

    ~TelemetrySocket() {
        if (sockfd >= 0) close(sockfd);
    }
};
```

```

}

// Initialize sender configuration (Onboard -> Ground)
bool initSender(const std::string& ip, int port) {
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) return false;

    memset(&target_addr, 0, sizeof(target_addr));
    target_addr.sin_family = AF_INET;
    target_addr.sin_port = htons(port);
    inet_pton(AF_INET, ip.c_str(), &target_addr.sin_addr);
    return true;
}

// Initialize receiver configuration (Ground listening port)
bool initReceiver(int port) {
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) return false;

    memset(&local_addr, 0, sizeof(local_addr));
    local_addr.sin_family = AF_INET;
    local_addr.sin_addr.s_addr = INADDR_ANY;
    local_addr.sin_port = htons(port);

    if (bind(sockfd, (struct sockaddr*)&local_addr, sizeof(
        local_addr)) < 0) {
        return false;
    }
    return true;
}

// Transmit telemetry frame data
bool sendData(const TelemetryData& data) {
    ssize_t sent = sendto(sockfd, &data, sizeof(TelemetryData), 0,
        (struct sockaddr*)&target_addr, sizeof(
            target_addr));
    return sent == sizeof(TelemetryData);
}

// Block until frame data is received
bool receiveData(TelemetryData& data) {
    socklen_t len = sizeof(target_addr);
    ssize_t rec = recvfrom(sockfd, &data, sizeof(TelemetryData), 0,
        (struct sockaddr*)&target_addr, &len);
    return rec == sizeof(TelemetryData);
}
};

```